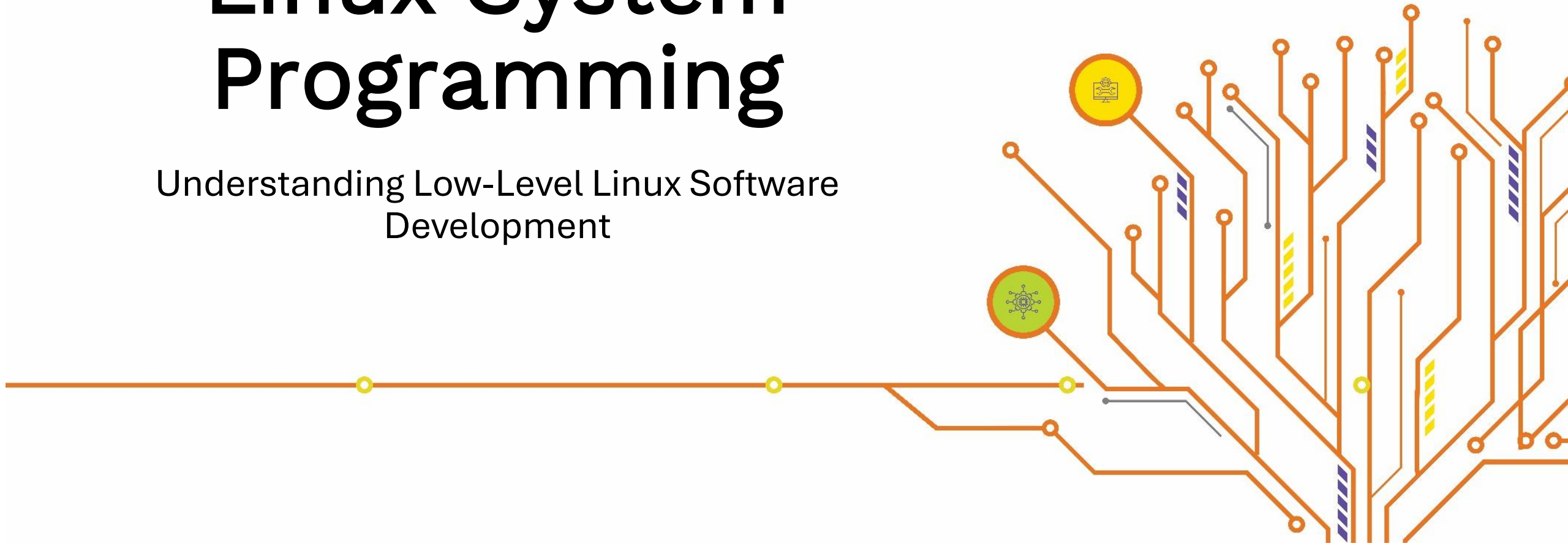
KPIT | APEX LAB

Introduction to Linux System Programming

Understanding Low-Level Linux Software Development



Disclaimer

This material has been developed exclusively for educational and academic purposes under the KPIT APEX Lab – Academic CSR Initiative. It is intended strictly for student learning, demonstrations, and classroom use.

Users are advised to verify and confirm the correctness of the contents before use.

It must not be copied, forwarded, shared, reproduced, or redistributed to any other individuals, groups, institutions, or platforms without prior written permission.

All examples, commands, diagrams, and demonstrations included here are meant solely to support conceptual understanding and learning.

Any use of these contents in commercial products, paid workshops, training programs, or proprietary documentation is strictly prohibited.

What is System Software?

system software is the foundational layer of software

Core software that manages hardware and provides fundamental services.

- enables the operating system, hardware, and core execution environment to function
- provides the essential services upon which application software depends
- typically includes components that interface closely with the kernel and C library and provide low-level services to applications

Application Software
GUI, scripts, tools

System Software
OS Kernel, Drivers, Libraries

Hardware
CPU, Memory, IO

Example of System Software

Examples

- **GUI frameworks / window systems -** X11, Wayland, Weston, GTK core services
- **Compiler toolchain -** GCC/Clang, assembler, linker (binutils)
- **Debugger -** GDB, LLDB
- **System daemons -** init systems, udev, dbus
- **Web server-** Nginx, Apache (provides infrastructure services to other apps)
- **Database engines -** MySQL, SQLite, PostgreSQL (foundational services used by applications)

System Software Vs Application Software

System software:

- Avoids large frameworks (Qt, Java, Node.js, .NET)
- Uses low-level APIs (syscalls, POSIX, libc)
- Often deals with raw data, hardware access, and OS internals
- Example: Writing a device access program using open()

Application software:

- Uses GUI toolkits, web frameworks, networking libraries
- Relies on abstracted APIs and toolkits that hide OS details
- Example: Python's requests library

System Software Does NOT Use Higher-Level Libraries

System Software Works With Fewer Abstractions

System software must understand:

- Memory layout
- Processes, threads, scheduling
- Filesystem internals
- Drivers and hardware behavior
- Networking stack primitives

Application path → high-level libraries → kernel

Applications, by contrast, get simplified abstractions:

- High-level I/O
- Framework-provided networking
- GUI abstraction layers
- Scripting languages

System program path → libc → syscalls → kernel

System Programming

- System programming is the discipline used to create system software.
- Writing software that interacts with system software and the OS kernel.
- Uses system calls to control processes, memory, I/O, and devices.
- Typically written in C/C++
- Close to hardware, performance-critical, and resource-aware.

System Programming → Creates System Software → Supports Application Software

System Programming

- System software interfaces with the kernel, libc, and low-level OS mechanisms, its development requires **system programming skills** such as:
  - Writing C programs that use POSIX APIs and system calls
  - Understanding Linux kernel behaviour
  - Managing memory manually
  - Handling low-level concurrency (threads, mutexes, semaphores)
  - Interacting with hardware through drivers or /dev

Why System Programming?

Performance: direct, efficient control over CPU, memory, I/O.

Determinism: required for embedded, automotive, real-time workloads.

Hardware access: reading sensors, controlling motors, talking to peripherals.

Creating foundational software: kernels, drivers, daemons, system libraries.

Better debugging and root-cause analysis: understand how the OS behaves.

automotive ECUs require deterministic behavior → system programming essential

Key Areas of System Programming

- Process creation and management
- File and device I/O
- Multithreading & synchronization
- Memory allocation & mapping
- Interprocess communication (IPC)
- Networking (sockets)
- Signals & timers
- Error handling and recovery

Cornerstones of System Programming

System Calls (syscalls)

C Library (libc)

C Compiler/linker (toolchain)

App → libc → syscall → kernel

libc translates C API → syscall ABI

System Calls

- System software interacts directly with the Linux kernel through:
  - **System calls** (ex: open(), read(), write())
  - **Device interfaces** (/dev, sysfs, procfs)
  - **Process, memory, and file operations**
- This layer understands how to communicate with kernel primitives rather than relying on high-level abstractions.

C Library (libc)

System software relies heavily on the **standard C library** (glibc, musl, uClibc), which provides:

- System call wrappers
- Memory allocation (malloc, free)
- Process control (fork, exec)
- File I/O (read, write)
- The C library forms the API boundary between user space and the kernel.

libc implements POSIX API, not all system calls

API — Application Programming Interface

“What the programmer sees.”

- An **API** is the *interface you program against*.

Examples

- POSIX API (open(), read(), write(), fork())
- C library functions (printf(), malloc())
- Driver user-space interfaces (ioctl, /sys, /proc)
- Linux system call interface *as declared in user-space headers*

ABI — Application Binary Interface

- An **ABI** describes *how compiled programs interact at the binary level*.
- It covers how **functions, data structures, calling conventions, and binaries** are laid out in memory.

“What the machine sees.”

Examples of ABI Elements

- How function parameters are passed (registers/stack)
- Binary format (ELF)
- Instruction set architecture (ARM, ARM64, x86)
- Symbol names (mangling)
- Struct memory layout (alignment, padding)
- System call numbers
- Shared library versioning (libc.so.6)

API vs ABI

Aspect	API	ABI
Visible to	Programmer	Machine / binary loader
Defined by	Headers, documentation	Compiler, linker, architecture
Enforced at	Compile time	Runtime
Stability needed for	Rebuilding code	Running already-compiled binaries
Breakage effect	Needs recompilation	Existing binaries break
Examples	POSIX, glibc functions	ELF format, calling conventions, system call numbers

Why API & ABI Matter?

- Cross-compiling
- Different CPU architectures = different ABIs.
- Shared libraries
- You can update libc, libm, or driver libraries, and all apps continue working — **only because ABI compatibility is maintained**.
- Kernel–User Space Interaction
- Syscalls are part of the **kernel ABI**
- Compiler / Toolchain
- The toolchain is built around a specific ABI:
- GCC configuration (--with-abi, --with-arch)
- glibc configuration
- Linker, assembler rules

POSIX - Portable Operating System Interface

- C APIs for Unix-like operating systems
- IEEE Std 1003.1-1990
- ISO/IEC 9945-1:1990
- POSIX defines **C code API** for many concepts
  - File and directory operations
  - Clocks and timers
  - Semaphores
  - Shared memory
  - Threads

- Official list of headers: http://pubs.opengroup.org/onlinepubs/9699919 799/idx/head.html
- Some familiar ones: <stdio.h>, <stdint.h>

Summary

- System programming = low-level control
- Uses syscalls + libc
- API vs ABI distinction
- POSIX standard